CLASE 5 — MACHINE LEARNING: FLUJO GENERAL Y APRENDIZAJE SUPERVISADO

5.1. Introducción: de los datos al aprendizaje

En las clases anteriores nos concentramos en comprender los datos. Aprendimos a observarlos, analizarlos, detectar problemas de calidad y prepararlos para que puedan ser utilizados posteriormente. Este recorrido no fue accidental. Antes de intentar construir un modelo de Inteligencia Artificial necesitamos conocer aquello que vamos a entregarle como información de entrada. Un algoritmo de Machine Learning puede ser matemáticamente correcto y, sin embargo, producir resultados deficientes si aprende a partir de datos incompletos, inconsistentes o mal preparados.

Ahora estamos en condiciones de dar un paso fundamental: comenzar a estudiar **Machine Learning** propiamente dicho.

La idea central de Machine Learning puede expresarse de una manera sencilla: en lugar de programar explícitamente todas las reglas que debe seguir un sistema para resolver un problema, construimos procedimientos capaces de **aprender patrones a partir de datos**.

Esta diferencia representa un cambio importante respecto de la programación tradicional. En un programa convencional, el programador analiza el problema, determina las reglas necesarias y las transforma en instrucciones que la computadora ejecutará. Si queremos determinar si una compra debe considerarse sospechosa, por ejemplo, podríamos escribir reglas como "si el importe supera determinado valor y la compra proviene de determinada ubicación, marcarla". En ese escenario, somos nosotros quienes definimos explícitamente las condiciones.

En Machine Learning el enfoque cambia. Podemos disponer de miles o millones de operaciones históricas en las que conocemos cuáles fueron legítimas y cuáles fueron fraudulentas. En lugar de escribir manualmente todas las reglas, utilizamos esos ejemplos para que un algoritmo encuentre relaciones que permitan construir un modelo capaz de realizar predicciones sobre nuevos casos.

La computadora sigue ejecutando instrucciones, por supuesto. La diferencia es **qué estamos programando**. No programamos directamente todas las reglas necesarias para responder cada caso particular; programamos un procedimiento de aprendizaje que, utilizando los datos disponibles, construirá un modelo.

Esta diferencia constituye uno de los conceptos fundamentales de toda la asignatura.

5.2. Programación tradicional y Machine Learning

Podemos comparar ambos enfoques mediante una situación concreta.

Supongamos que una empresa quiere determinar si un cliente probablemente realizará una compra durante su próxima visita.

En una solución tradicional podríamos establecer reglas:

Si visitó el sitio más de 5 veces

y agregó productos al carrito

y tiene una cuenta registrada

entonces → probablemente compre.

El problema aparece rápidamente. ¿Qué ocurre con un cliente que visitó el sitio solamente tres veces pero compró anteriormente productos de alto valor? ¿Y con uno que agregó productos al carrito pero abandonó varias veces? ¿Qué sucede si descubrimos que la hora del día, el dispositivo utilizado o el tipo de producto también influyen?

A medida que intentamos cubrir todas las situaciones posibles, las reglas comienzan a crecer y hacerse difíciles de mantener.

En Machine Learning podemos utilizar registros históricos como:

Visitas	**Carrito**	**Compras previas**	**Gasto**	**¿Compró?**
6	Sí	4	250000	Sí
2	No	0	0	No
8	Sí	1	80000	Sí
3	Sí	0	0	No

10	Sí	7	500000	Sí

El algoritmo puede utilizar esos ejemplos para aprender una relación entre las características de los clientes y el resultado observado.

Cuando aparezca un cliente nuevo, el sistema podrá recibir sus características y producir una predicción.

El flujo conceptual sería:

DATOS HISTÓRICOS

↓

APRENDIZAJE

↓

MODELO

↓

NUEVO CLIENTE

↓

PREDICCIÓN

La palabra **aprendizaje** puede resultar engañosa si la interpretamos como si la computadora comprendiera el problema de la misma manera que una persona. El algoritmo no "entiende" el negocio como un ser humano. Lo que hace es ajustar una estructura matemática utilizando los datos disponibles para representar determinados patrones.

Esta distinción será importante durante toda la materia: **Machine Learning no significa que la computadora piense como una persona; significa que un procedimiento computacional puede ajustar un modelo a partir de datos para realizar determinadas tareas.**

5.3. ¿Qué significa aprender?

Cuando decimos que un modelo "aprende" estamos hablando de un proceso matemático.

Supongamos que queremos predecir el precio de una vivienda utilizando su superficie.

Podríamos plantear un modelo muy sencillo:

precio=w⋅superficie+bprecio = w \cdot superficie + b

donde ww representa cuánto influye la superficie y bb representa un término independiente denominado intercepto o bias, dependiendo del contexto del modelo.

Inicialmente no conocemos los valores adecuados de ww y bb. El algoritmo utiliza los datos disponibles para encontrar valores que permitan realizar predicciones razonablemente cercanas a los precios observados.

Por ejemplo, podríamos comenzar con:

w=100w=100

y obtener predicciones muy alejadas de los valores reales.

El proceso de entrenamiento irá modificando los valores internos del modelo buscando una configuración que produzca un menor error de acuerdo con el criterio definido.

Por eso podemos decir que el modelo **aprende sus parámetros a partir de los datos**.

Este punto nos permite introducir una distinción que será fundamental en las próximas clases: **algoritmo, modelo y parámetros no son exactamente lo mismo**.

5.4. Algoritmo y modelo

En Machine Learning utilizamos algoritmos de aprendizaje para construir modelos.

Podemos pensar el algoritmo como el procedimiento que indica **cómo aprender** y el modelo como la estructura resultante de ese aprendizaje.

Por ejemplo, la regresión lineal utiliza un procedimiento matemático para encontrar los coeficientes que mejor representan una relación entre las variables de entrada y la variable objetivo.

Una vez entrenada, la regresión lineal podría quedar representada por:

y=2.7x+15y=2.7x+15

La expresión concreta constituye el modelo obtenido después del entrenamiento.

El algoritmo fue utilizado para encontrar los valores adecuados de los parámetros.

Esta diferencia puede parecer sutil, pero es fundamental. Cuando entrenamos un modelo no estamos simplemente "ejecutando un algoritmo". Estamos utilizando un algoritmo de aprendizaje sobre determinados datos para obtener una instancia concreta de un modelo.

Podemos visualizarlo de esta manera:

DATOS + ALGORITMO

↓

ENTRENAMIENTO

↓

MODELO

↓

PREDICCIÓN

Si cambiamos los datos, podríamos obtener un modelo diferente. Si cambiamos los hiperparámetros, también podríamos obtener un modelo diferente. Incluso utilizando el mismo algoritmo, dos entrenamientos pueden producir resultados distintos.

5.5. Parámetros

Los **parámetros** son valores internos que el modelo aprende durante el proceso de entrenamiento.

Volvamos a la ecuación:

$y=wx+by=wx+b$

Los valores w y b son parámetros del modelo.

No somos nosotros quienes decidimos manualmente cuál debe ser su valor final. El proceso de entrenamiento los ajusta utilizando los datos.

Esta característica diferencia a los parámetros de los hiperparámetros.

Es importante que el estudiante pueda responder una pregunta aparentemente sencilla:

¿Quién determina el valor de un parámetro?

La respuesta general es: **el proceso de entrenamiento**.

Cuando entrenamos una regresión lineal, por ejemplo, el algoritmo determina los coeficientes que mejor se ajustan a los datos según el criterio utilizado.

Cuando entrenamos una red neuronal, los pesos y bias se modifican durante el entrenamiento.

Cuando entrenamos otros modelos, los parámetros pueden adoptar formas diferentes.

Por eso no debemos pensar que todos los modelos tienen los mismos parámetros. Cada familia de modelos posee su propia estructura matemática.

5.6. Hiperparámetros

Los **hiperparámetros**, en cambio, son configuraciones que establecemos antes o durante el proceso de entrenamiento y que controlan cómo se construye o aprende el modelo.

Por ejemplo, en KNN debemos decidir el valor de KK, que determina cuántos vecinos se utilizan para realizar una predicción.

En un árbol de decisión podemos establecer una profundidad máxima.

En Random Forest podemos definir cuántos árboles utilizar.

En una red neuronal podemos decidir la tasa de aprendizaje, la cantidad de épocas o el tamaño del batch.

Estos valores no son aprendidos de la misma manera que los parámetros internos.

Podemos resumir la diferencia:

Concepto	¿Quién lo determina?	Ejemplo
Parámetro	El entrenamiento	pesos de una red, coeficientes

Hiperparámetro	Configuración del entrenamiento	K, profundidad, learning rate

Esta diferencia será especialmente importante cuando lleguemos a la experimentación. En Machine Learning no basta con entrenar un modelo una sola vez. Muchas veces debemos probar distintas configuraciones para determinar cuál funciona mejor.

5.7. Features y target

Para comenzar a construir un problema de Machine Learning necesitamos identificar qué información utilizaremos como entrada y qué queremos obtener como resultado.

Las variables utilizadas como información de entrada suelen denominarse **features**.

La variable que queremos predecir en un problema supervisado se denomina **target** o variable objetivo. Cuando hablamos específicamente de clasificación también es frecuente utilizar el término **label**.

Supongamos nuevamente nuestro problema de e-commerce.

Queremos predecir:

> ¿Este usuario realizará una compra?

Podríamos utilizar como features:

edad

cantidad de visitas

productos vistos

cantidad de productos agregados al carrito

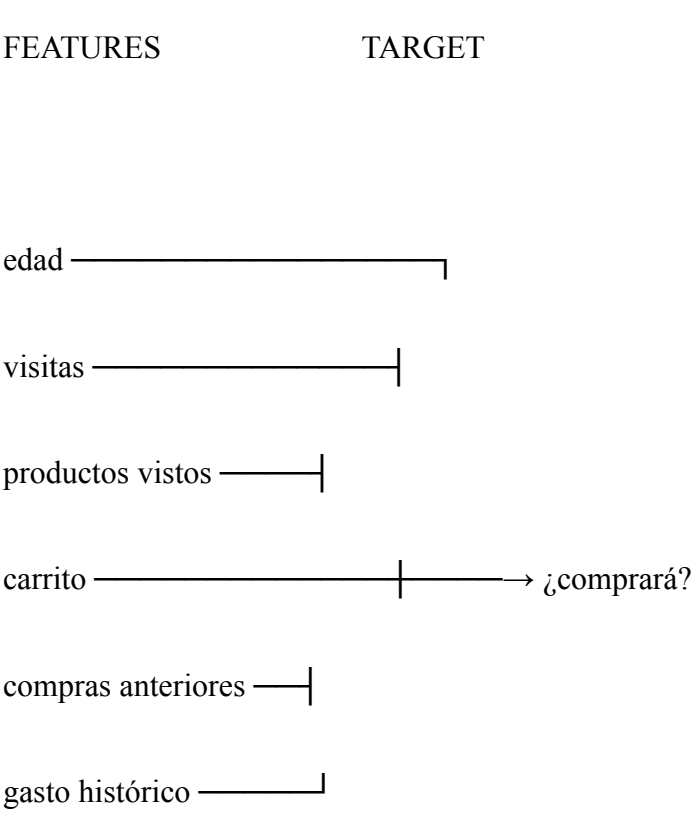
compras anteriores

gasto histórico

Y como target:

compró → sí / no

Podemos representarlo:



Esta separación es fundamental porque el modelo debe aprender utilizando información que estaría disponible **antes de realizar la predicción**.

Y aquí aparece uno de los problemas más importantes que debemos aprender a detectar: el **data leakage**.

5.8. Data leakage

Supongamos que queremos predecir si un cliente comprará un producto.

Entre las variables disponibles aparece:

fecha_compra

Si utilizamos esa variable para predecir una compra que todavía no ocurrió, estamos introduciendo información que en realidad no estaría disponible en el momento de la predicción.

El modelo podría aprender una relación extraordinariamente buena porque estamos entregándole información del futuro.

Otro ejemplo sería utilizar una variable denominada:

monto_compra

para predecir si el cliente realizará esa misma compra.

El modelo tendría prácticamente la respuesta delante suyo.

El resultado podría parecer excelente durante la evaluación, pero sería completamente engañoso.

Esto es **data leakage**: información que no debería estar disponible para el modelo durante la predicción termina formando parte de las variables utilizadas para entrenarlo o evaluarlo.

La importancia de este concepto crecerá cuando dividamos los datos y construyamos nuestro flujo completo de entrenamiento.

5.9. Instancias

Cada observación individual de nuestro dataset puede considerarse una **instancia**.

Si tenemos una tabla donde cada fila representa un cliente, cada fila constituye una instancia.

Si cada fila representa una transacción, cada fila es una instancia de una transacción.

Por ejemplo:

Cliente 1024

edad = 31

visitas = 7

carrito = 1

gasto = 185000

Ese registro constituye una instancia.

Si tenemos 5.000 filas, tendremos 5.000 instancias, siempre que cada fila represente una observación independiente según la estructura definida.

Esta aclaración es importante porque el número de filas de una tabla no siempre coincide conceptualmente con el número de personas. Podemos tener 5.000 transacciones realizadas por solamente 800 clientes.

La interpretación depende del diseño del dataset.

5.10. El entrenamiento

El entrenamiento es el proceso mediante el cual utilizamos los datos disponibles para ajustar los parámetros del modelo.

Podemos imaginar que tenemos:

Features → $[x_1, x_2, x_3, ...]$

Target → y

El algoritmo recibe los ejemplos y busca construir una relación entre las features y el target.

En términos simplificados:

$f(X) \approx y$f(X)\approx y

donde X representa las variables de entrada y y el resultado que queremos predecir.

Durante el entrenamiento, el algoritmo busca una configuración del modelo que permita obtener predicciones suficientemente buenas sobre los datos utilizados para aprender.

Pero aquí aparece un problema que será central en las próximas clases.

¿Qué significa que una predicción sea "buena"?

¿Y qué significa que un modelo que funciona muy bien con los datos que utilizamos para entrenarlo también vaya a funcionar bien con clientes que nunca había visto?

Estas preguntas nos llevan directamente a la evaluación y a la generalización.

5.11. Entrenamiento no significa memorizar

Un error conceptual frecuente consiste en pensar que el objetivo del entrenamiento es conseguir que el modelo reproduzca perfectamente los datos utilizados.

En realidad, buscamos que el modelo aprenda **patrones que puedan generalizarse a datos nuevos**.

Supongamos que tenemos 1.000 ejemplos históricos.

Un modelo extremadamente flexible podría aprender prácticamente todos esos ejemplos y alcanzar un rendimiento excelente sobre ellos.

Pero si cuando recibe un ejemplo nuevo comete muchos errores, entonces no hemos construido un buen modelo.

El objetivo no es recordar los datos.

El objetivo es **aprender una relación útil que pueda aplicarse a observaciones que todavía no conocemos**.

Esta idea de generalización será uno de los ejes centrales de la Clase 7, donde estudiaremos formalmente overfitting, underfitting, validación y métricas.

5.12. Inferencia y predicción

Una vez entrenado el modelo podemos utilizarlo sobre nuevos datos.

Esta etapa suele denominarse **inferencia** o predicción.

Supongamos que entrenamos un modelo utilizando miles de clientes históricos.
Posteriormente aparece un cliente nuevo:

edad = 29

visitas = 8

carrito = 1

compras_previas = 2

gasto = 170000

El modelo recibe esas características y produce una predicción:

probablemente compra

En un modelo de clasificación también podríamos obtener una probabilidad:

P(compra) = 0.82

y luego aplicar un criterio de decisión para determinar la clase final.

La diferencia entre entrenamiento e inferencia debe quedar muy clara:

durante el entrenamiento el modelo aprende sus parámetros; durante la inferencia utilizamos el modelo ya entrenado para producir una predicción.

Esta distinción será especialmente importante cuando estudiemos redes neuronales.

5.13. El flujo completo de Machine Learning

Hasta ahora hemos introducido varios conceptos que pueden parecer independientes. En realidad forman parte de un único proceso.

Un proyecto de Machine Learning comienza con un problema.

No comenzamos preguntando qué algoritmo queremos utilizar. Primero debemos determinar qué queremos resolver.

El flujo general puede representarse así:

PROBLEMA

↓

DATOS

↓

PREPARACIÓN

↓

FEATURES / TARGET

↓

TRAIN / VALIDATION / TEST

↓

ENTRENAMIENTO

↓

PREDICCIÓN

↓

EVALUACIÓN

↓

EXPERIMENTACIÓN

↓

SELECCIÓN

↓

MODELO FINAL

Este esquema debería convertirse en uno de los mapas conceptuales principales de la materia.

La razón es que muchas veces cuando se comienza a estudiar Machine Learning se produce una confusión: se presentan numerosos algoritmos y el alumno termina asociando Machine Learning con "elegir un algoritmo y ejecutarlo".

El proceso real es bastante más amplio.

Elegir un algoritmo es solamente una parte.

5.14. El problema antes que el algoritmo

Imaginemos que una empresa nos dice:

> "Queremos utilizar Inteligencia Artificial para mejorar nuestras ventas."

Esta afirmación todavía no constituye un problema de Machine Learning suficientemente definido.

Necesitamos convertirla en una pregunta concreta.

Podríamos plantear:

> ¿Podemos predecir si un usuario realizará una compra durante su próxima sesión?

Ahora tenemos una tarea que puede transformarse en un problema supervisado de clasificación.

También podríamos plantear:

> ¿Podemos estimar cuánto gastará un cliente durante el próximo mes?

En este caso tendríamos un problema de regresión.

O:

> ¿Podemos agrupar clientes según sus comportamientos de compra?

Aquí podríamos estar frente a un problema de aprendizaje no supervisado.

La elección del enfoque depende entonces del problema y de la información disponible.

No deberíamos comenzar diciendo:

> "Voy a usar Random Forest."

Deberíamos comenzar preguntando:

> "¿Qué quiero predecir o descubrir?"

Esta forma de razonar será una constante durante el resto de la asignatura.

5.15. Los tres grandes paradigmas

Hasta este momento hablamos principalmente de problemas en los que disponemos de ejemplos históricos y conocemos el resultado que queremos aprender.

Ese escenario corresponde al **aprendizaje supervisado**.

Pero Machine Learning no se limita a él.

Podemos distinguir tres grandes paradigmas que estudiaremos durante la materia:

aprendizaje supervisado, **aprendizaje no supervisado** y **aprendizaje por refuerzo**.

La diferencia fundamental está relacionada con la información que recibe el algoritmo y con el objetivo que buscamos alcanzar.

En el aprendizaje supervisado disponemos de ejemplos acompañados por una respuesta conocida.

Podemos representarlo:

datos + respuesta correcta

↓

aprendizaje

↓

predicción

Si tenemos clientes y sabemos cuáles compraron, podemos entrenar un modelo para predecir la compra de nuevos clientes.

En el aprendizaje no supervisado no disponemos de una variable objetivo que indique cuál debería ser la respuesta.

Tenemos datos y queremos descubrir estructuras.

Por ejemplo:

clientes

↓

características

↓

algoritmo

↓

grupos descubiertos

El algoritmo puede encontrar grupos de clientes con comportamientos similares sin que nosotros le hayamos indicado previamente cuáles son las categorías.

Finalmente tenemos el aprendizaje por refuerzo.

Aquí el escenario cambia nuevamente. Tenemos un agente que interactúa con un entorno, realiza acciones y recibe recompensas o penalizaciones.

El esquema general será:

ESTADO

↓

ACCIÓN

↓

ENTORNO

↓

RECOMPENSA

↓

NUEVO ESTADO

Este paradigma será estudiado al final de la materia.

5.16. Aprendizaje supervisado

El aprendizaje supervisado es probablemente el paradigma que más utilizaremos inicialmente porque permite trabajar directamente con problemas de predicción.

La palabra "supervisado" hace referencia a que durante el entrenamiento disponemos de ejemplos donde conocemos la respuesta correcta.

Supongamos:

Edad	Visitas	Gasto	Compra
22	2	0	No
35	8	180000	Sí
41	5	120000	Sí
19	1	0	No

El modelo recibe las características y conoce el resultado histórico.

Durante el entrenamiento intenta aprender una relación:

X→yX\rightarrow y

Luego, ante un nuevo cliente:

edad = 30

visitas = 6

gasto = 100000

el modelo debe producir una predicción.

La respuesta correcta de ese cliente todavía puede no estar disponible. Precisamente por eso queremos predecirla.

5.17. Clasificación y regresión

Dentro del aprendizaje supervisado aparecen dos grandes tipos de problemas que serán fundamentales en las próximas clases: **clasificación y regresión**.

En la regresión, el target representa un valor numérico.

Podemos querer predecir:

precio

gasto mensual

cantidad de ventas

tiempo de entrega

Por ejemplo:

precio=125000precio=125000

La salida es un número.

En clasificación, en cambio, queremos determinar una categoría.

Por ejemplo:

compra → Sí / No

o:

cliente → bajo / medio / alto

o:

producto → categoría A / B / C

La salida pertenece a una clase.

La diferencia puede parecer sencilla, pero modifica profundamente la manera en que evaluaremos los modelos.

Si intentamos predecir un precio de 100.000 y obtenemos 98.000, podemos medir la diferencia entre ambos valores.

Si queremos determinar si un cliente comprará y el modelo responde "No" cuando realmente compró, tenemos un error de clasificación.

En las siguientes clases estudiaremos modelos específicos para ambos tipos de problemas.

5.18. Transformar el e-commerce en un problema de Machine Learning

Ahora podemos aplicar todo lo visto al proyecto transversal.

Tenemos nuestro dataset CLEAN y queremos comenzar a utilizarlo para Machine Learning.

La primera pregunta no debería ser qué algoritmo ejecutar.

Supongamos que decidimos trabajar con el problema:

Predecir si un usuario realizará una compra.

Tenemos que identificar primero las variables disponibles.

Podríamos tener:

edad

cantidad_visitas

productos_vistos

productos_carrito

compras_previas

gasto_historico

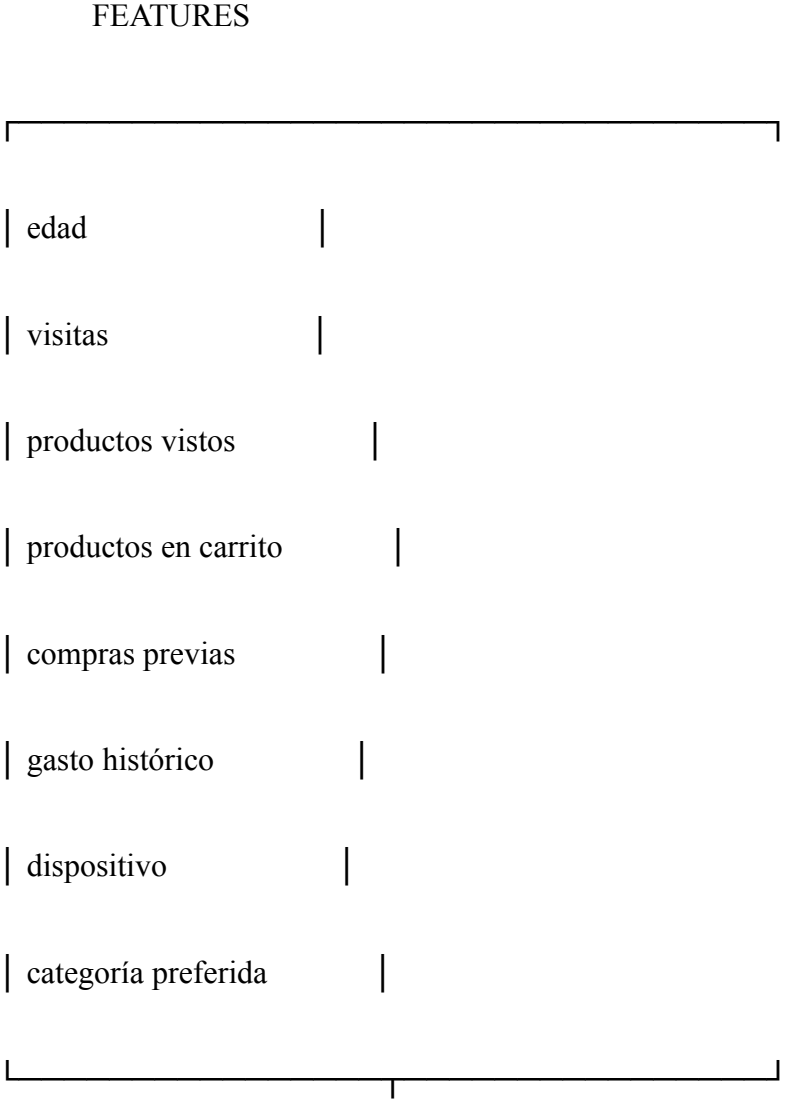
dispositivo

categoria_preferida

Y como target:

compra

Nuestro problema queda entonces representado como:



SÍ / NO

En este punto todavía no hemos elegido un algoritmo.

Y eso está bien.

Primero definimos correctamente el problema.

Después analizaremos qué modelos pueden ser apropiados.

5.19. ¿Todas las variables sirven?

Una de las decisiones más importantes consiste en determinar qué variables realmente deben ingresar al modelo.

Supongamos que nuestro dataset tiene:

id_cliente

fecha_compra

monto_compra

cantidad_visitas

productos_vistos

edad

No necesariamente todas deben utilizarse como features.

`id_cliente`, por ejemplo, puede ser simplemente un identificador. Que un cliente tenga ID 4587 no significa que tenga una característica útil para predecir su comportamiento.

`fecha_compra` podría contener información del futuro dependiendo de cómo definimos el problema.

`monto_compra` podría ser directamente consecuencia del evento que queremos predecir.

Esto muestra que la selección de variables no consiste solamente en elegir todas las columnas disponibles.

Debemos preguntarnos:

> **¿Esta variable representa información que estaría disponible en el momento real en que quiero realizar la predicción?**

Si la respuesta es no, utilizarla puede producir leakage.

Esta pregunta será especialmente importante cuando construyamos nuestros conjuntos de entrenamiento, validación y prueba.

5.20. Separar los datos

Supongamos que tenemos 5.000 observaciones.

Podríamos utilizar todas ellas para entrenar un modelo.

Pero entonces surge una pregunta:

> ¿Cómo sabemos si funciona bien con datos que nunca vio?

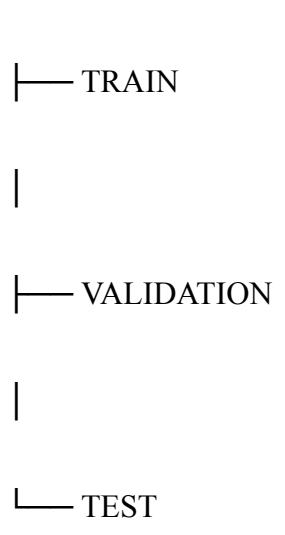
Si evaluamos el modelo sobre los mismos ejemplos que utilizó para aprender, estaremos midiendo principalmente qué tan bien se ajustó a esos datos.

No estaremos midiendo adecuadamente su capacidad para generalizar.

Por eso necesitamos separar los datos.

Una representación conceptual puede ser:

DATASET



El conjunto de **training** se utiliza para entrenar.

El conjunto de **validation** se utiliza durante el proceso de selección y ajuste del modelo.

El conjunto de **test** se reserva para realizar una evaluación final.

La idea fundamental es que el conjunto de prueba debe mantenerse separado del proceso de ajuste.

Si miramos repetidamente el test y modificamos nuestro modelo basándonos en sus resultados, estamos utilizando indirectamente esa información para tomar decisiones. En consecuencia, deja de funcionar como una evaluación verdaderamente independiente.

Este concepto será desarrollado con mucha mayor profundidad en la Clase 7.

5.21. Validación y experimentación

Machine Learning es un proceso experimental.

Rara vez construimos un único modelo, lo entrenamos una vez y concluimos inmediatamente que es la mejor solución.

Podemos tener:

Modelo A

Modelo B

Modelo C

y diferentes configuraciones para cada uno.

Podemos modificar hiperparámetros, características, transformaciones y otros aspectos del proceso.

Pero cada modificación debe evaluarse de manera controlada.

Por ejemplo:

Modelo	Configuración	Validation

KNN	K=3	0,78
KNN	K=7	0,82
Árbol	profundidad=3	0,80
Árbol	profundidad=8	0,76

La tabla no es solamente una forma de registrar resultados. Representa una parte esencial del trabajo científico y técnico con Machine Learning.

Estamos formulando configuraciones, realizando experimentos, observando resultados y tomando decisiones.

Más adelante incorporaremos métricas más completas y analizaremos por qué un modelo con una determinada precisión puede ser menos conveniente que otro.

5.22. Generalización

Llegamos ahora a uno de los conceptos más importantes de esta clase.

Un modelo debe ser capaz de **generalizar**.

Generalizar significa que aquello que aprendió a partir de los datos de entrenamiento pueda utilizarse razonablemente bien sobre datos nuevos.

Imaginemos un estudiante que memoriza exactamente las respuestas de un examen de práctica. Si luego recibe exactamente las mismas preguntas, puede obtener una calificación perfecta.

Pero si el examen cambia ligeramente y requiere aplicar los conceptos aprendidos, puede fracasar.

Algo parecido puede ocurrir con un modelo de Machine Learning.

Si simplemente memoriza patrones específicos de los datos de entrenamiento, puede obtener un desempeño excelente sobre ellos pero fallar ante nuevos ejemplos.

Por eso no buscamos simplemente minimizar el error de entrenamiento.

Buscamos construir un modelo que encuentre patrones suficientemente generales.

Esta idea nos llevará posteriormente al concepto de **overfitting**.

5.23. Underfitting y overfitting: primera aproximación

Aunque profundizaremos en estos conceptos durante la clase dedicada a evaluación, es importante introducirlos ahora porque forman parte de la lógica del aprendizaje.

El **underfitting** ocurre cuando el modelo resulta demasiado simple para representar adecuadamente los patrones presentes en los datos.

Imaginemos que queremos representar una relación claramente curva utilizando un modelo lineal extremadamente simple. El modelo puede ser incapaz de capturar la estructura existente.

En ese caso tendremos un modelo que no aprende suficientemente.

El **overfitting** aparece en el extremo contrario.

El modelo tiene una complejidad suficiente para ajustarse demasiado a los datos de entrenamiento y puede terminar aprendiendo particularidades que no representan patrones generales.

Conceptualmente:

Modelo demasiado simple

↓

UNDERFITTING

Modelo adecuado

↓

GENERALIZACIÓN

Modelo demasiado complejo

↓

OVERFITTING

Este equilibrio entre capacidad del modelo y generalización será central cuando estudiemos árboles, KNN, SVM, redes neuronales y otros algoritmos.

5.24. ¿Cómo sabemos si un modelo es bueno?

Esta pregunta parece sencilla, pero no tiene una respuesta única.

Supongamos que construimos un modelo para detectar operaciones fraudulentas.

Tenemos 10.000 transacciones y solamente 100 son fraudulentas.

Un modelo extremadamente ingenuo podría predecir:

"todas son legítimas"

Entonces acertaría 9.900 de 10.000.

Su accuracy sería:

990010000=0.99\frac{9900}{10000}=0.99

es decir, 99 %.

A primera vista parece un modelo excelente.

Pero no detectó **ninguna** operación fraudulenta.

Esto demuestra que evaluar un modelo requiere comprender el problema y elegir métricas apropiadas.

En clasificación tendremos que distinguir entre diferentes tipos de aciertos y errores. Por eso la próxima etapa del recorrido estará dedicada específicamente a clasificación y regresión, y posteriormente tendremos una clase completa dedicada a evaluación.

5.25. El papel del conocimiento del dominio

A medida que avanzamos aparece una idea que vale la pena destacar: Machine Learning no reemplaza automáticamente el conocimiento sobre el problema.

Si estamos trabajando con un e-commerce, necesitamos conocer qué representa una compra, qué significa una devolución, qué variables están disponibles antes de una transacción y qué comportamiento puede considerarse razonable.

El algoritmo puede encontrar patrones que nosotros no habíamos observado, pero necesitamos interpretar esos patrones y determinar si tienen sentido.

Por ejemplo, podría aparecer una correlación muy fuerte entre una variable aparentemente irrelevante y la compra.

Antes de asumir que descubrimos una característica importante deberíamos investigar si esa variable contiene información indirecta sobre el target o si existe algún problema de construcción del dataset.

Podría incluso tratarse de leakage.

Por eso un buen proyecto de Machine Learning combina:

datos

+

estadística

+

algoritmos

+

evaluación

+

conocimiento del dominio

Ninguno de estos elementos funciona correctamente aislado.

5.26. Machine Learning como proceso iterativo

Otra idea importante es que el flujo que presentamos anteriormente no debe interpretarse como una secuencia que se ejecuta una sola vez.

En realidad, el proceso suele ser iterativo.

Podemos comenzar con:

Problema

↓

Datos

↓

Preparación

↓

Modelo

↓

Evaluación

y descubrir durante la evaluación que algo no funciona correctamente.

Entonces podemos volver atrás.

Quizás encontramos que una variable está mal construida.

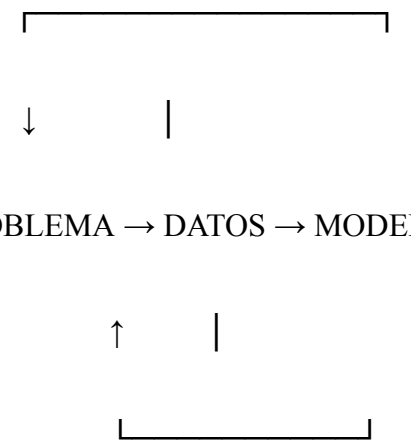
Quizás existen valores extremos.

Quizás el modelo presenta overfitting.

Quizás necesitamos modificar los hiperparámetros.

Quizás las features seleccionadas no contienen suficiente información.

Por eso el flujo real puede representarse como un ciclo:



La evaluación genera información que puede conducir a nuevos experimentos.

Este carácter iterativo es una de las razones por las cuales Machine Learning se parece más a un proceso experimental que a una simple ejecución de código.

5.27. Ejemplo integrador

Supongamos que una empresa de comercio electrónico quiere anticipar qué clientes tienen mayor probabilidad de realizar una compra.

Disponemos de registros históricos. Después del proceso de limpieza realizado en las clases anteriores tenemos un dataset preparado.

Primero definimos el problema:

> Predecir si un cliente realizará una compra.

Luego identificamos el target:

compra = Sí / No

Después seleccionamos las features disponibles antes de la compra:

edad

visitas

productos vistos

productos en carrito

compras previas

gasto histórico

Dividimos los datos en conjuntos destinados al entrenamiento, validación y prueba.

A continuación seleccionamos un algoritmo de clasificación y realizamos el entrenamiento.

El modelo aprende sus parámetros a partir del conjunto de entrenamiento.

Luego realizamos predicciones sobre datos que no utilizamos directamente para ajustar esos parámetros.

Evaluamos el desempeño.

Si los resultados no son satisfactorios, no necesariamente significa que "el algoritmo sea malo". Podemos revisar los datos, las variables, los hiperparámetros, la representación del problema o incluso la definición del target.

Si el modelo funciona bien, todavía debemos preguntarnos si generaliza adecuadamente y si la métrica utilizada realmente representa lo que necesita la empresa.

Finalmente, después de realizar los experimentos y seleccionar una configuración adecuada, podemos construir nuestro modelo final.

Este ejemplo reúne prácticamente todo el recorrido conceptual de la clase.

5.28. Cierre conceptual

A partir de esta clase podemos empezar a mirar Machine Learning de una manera diferente. Ya no deberíamos entenderlo como una colección de algoritmos independientes, sino como un proceso mediante el cual transformamos datos en modelos capaces de realizar predicciones o descubrir estructuras.

Partimos de un problema, identificamos los datos necesarios, preparamos la información, definimos las features y el target cuando corresponde, separamos los conjuntos de datos, entrenamos un modelo, realizamos predicciones, evaluamos sus resultados y experimentamos hasta encontrar una configuración adecuada.

También diferenciamos conceptos que suelen confundirse. El **algoritmo** es el procedimiento utilizado para aprender; el **modelo** es la estructura obtenida después del entrenamiento; los **parámetros** son valores que el modelo aprende; los **hiperparámetros** son configuraciones que controlan el proceso de aprendizaje.

Comprendimos además que el objetivo no consiste en memorizar los datos de entrenamiento, sino en construir modelos capaces de **generalizar** sobre observaciones nuevas. Esta idea nos

permitió introducir los conceptos de underfitting y overfitting, que desarrollaremos con profundidad más adelante.

Finalmente ubicamos el aprendizaje supervisado dentro de un panorama más amplio. Cuando tenemos respuestas conocidas podemos trabajar con clasificación o regresión. Cuando no disponemos de un target y queremos descubrir estructuras podemos utilizar aprendizaje no supervisado. Y cuando un agente interactúa con un entorno y aprende a partir de recompensas entramos en el aprendizaje por refuerzo.

Con esto tenemos construido el mapa conceptual que necesitamos antes de comenzar a estudiar modelos concretos.

La próxima clase dará el siguiente paso natural: **¿cómo aprende realmente un modelo supervisado y cómo podemos utilizar distintos modelos para resolver problemas de regresión y clasificación?**

Ahí entraremos en regresión lineal y polinómica, clasificación, KNN, árboles de decisión y regresión logística, pero siempre manteniendo como eje la misma pregunta que atraviesa toda esta materia:

> **¿Cómo transformamos datos en un modelo que pueda tomar decisiones útiles sobre casos que todavía no conocemos?**